

Original Article

An Investigation into a Formal Semantic Framework for Goal-Oriented Requirements Models

Trinh Hoang Viet La¹, Duc-Hanh Dang^{1*}

¹ Faculty of Information Technology, VNU University of Engineering and Technology, Hanoi, Viet Nam

Received DD Mon YYYY;

Revised DD Mon YYYY; Accepted DD Mon YYYY

Abstract: In goal-oriented requirements engineering (GORE), models such as iStar effectively capture stakeholders' intentions and dependency relationships. However, iStar is fundamentally declarative and centered on the actor–goal perspective; it provides only limited system-oriented structuring for explicitly modeling software behavior, and lacks a direct formal basis for expressing and analyzing progress and liveness properties. These limitations make it difficult to trace requirements toward formally analyzable specifications and hinder verification activities in critical systems. To address these issues, this paper proposes a verification-oriented approach based on gStar, an extension of iStar designed to strengthen system-side modeling capabilities and to standardize the information required for formalization. On this basis, we define a verification-oriented formal semantics for the considered gStar fragment and develop a systematic model-driven engineering (MDE) transformation process from requirements models to Event-B formal specifications. In addition, we propose a template-guided approach for constructing and checking Linear Temporal Logic (LTL) formulae from the model in order to specify safety and temporal progress properties, while also supporting requirement traceability in complementary verification activities. The proposed approach helps narrow the gap between declarative requirements models and verifiable system specifications, while preserving core semantic correspondences throughout the formalization process. Our contributions provide a foundation for the consistent formalization of goal-oriented requirements and support the systematic analysis of safety and progress properties in the transformed model.

Keywords: Goal-oriented requirements engineering, iStar, Event-B, model transformation, linear temporal logic

1. Introduction

In modern software engineering, many systems must address not only what should

be done, but also when it should be done, in what order it should be carried out, and how the system should react to changing environmental conditions. This is particularly important in contexts that require timely responses, coordination among multiple components, and

*Corresponding author.

E-mail address: hanhdd@vnu.edu.vn

operation in dynamic environments. In this context, Goal-Oriented Requirements Engineering (GORE) provides a useful foundation for modeling stakeholders' intentions, strategies, and dependency relationships. Among GORE approaches, iStar is one of the most widely used frameworks because of its intuitive and structured representation of goals, tasks, and dependencies [1].

However, the iStar community standard remains fundamentally declarative and centered on the actor–goal perspective. Its core language does not provide sufficiently system-oriented operational semantics to represent software behavior in terms of states and events, nor does it offer a direct temporal semantic basis for describing progress and liveness properties. This is consistent with the design scope of iStar 2.0, which emphasizes intentions and dependency relationships rather than time-aware operational specifications [1]. As a result, ensuring consistency and analyzability between requirements models and operational specifications remains a major challenge, especially in critical systems that require formal verification.

Many studies have attempted to increase the formal rigor of iStar models by mapping them to different formal languages. Some approaches translate iStar into Z in order to obtain mathematical specifications for analyzing constraints and correctness on the basis of Z's logical foundations [2]. Another line of work exploits Alloy for automated checking and counterexample finding with respect to model constraints [3]. In addition, ontology- or description-logic-based representations such as iStar2.0-OWL aim to standardize conceptual semantics and support reasoning [4]. These efforts are valuable for clarifying constraints, detecting inconsistencies, and standardizing semantics at the structural level. However, most of them remain focused on static consistency, type checking, or conceptual-level reasoning. By contrast, an end-

to-end bridge from requirements models to event-based operational behavior—where properties can be proved through proof obligations and analyzed over the state space through model checking—is still rather limited. This limitation becomes even more evident when contrasted with verification platforms such as Event-B/Rodin/ProB, which combine event-based modeling, proof obligations, and model checking within a single ecosystem [5, 6].

At the same time, the need to express uncertainty, adaptation, or domain-specific concerns has motivated several extensions of goal models. For example, FLAGS introduces fuzzy goals to support requirements-driven adaptation [7]. Extensions of iStar/GRL for collaborative cyber-physical systems (CPS) also enhance modeling capability in complex environments [8]. However, even when expressiveness is increased at the modeling level, there still lacks a systematic approach for normalizing system-oriented information, automatically generating verifiable operational specifications, and integrating these steps into a unified verification pipeline supported by a well-established proof- and model-checking toolchain.

Meanwhile, Event-B and the Rodin/ProB ecosystem provide a strong formal-methods foundation for invariant-based development, proof obligations, and model checking. Rodin supports the generation and discharge of proof obligations for invariants [5]; ProB supports animation and model checking for detecting counterexamples, deadlocks, and exploring the state space [6]. Therefore, if goal-oriented requirements models can be systematically transformed into Event-B specifications in a traceable and semantically grounded manner, then a unified analysis mechanism can be obtained for both requirements consistency and system behavior. In addition, Linear Temporal Logic (LTL) provides a classical foundation for specifying safety and temporal progress properties [9]. However, using LTL

as an additional validation layer for goal models requires clear binding and traceability between goal/task/dependency structures on the requirements side and the corresponding state variables in the behavioral model.

To address these limitations, we propose a verification-oriented approach for improving the consistency and analyzability of operationalized goal models. The approach consists of two steps: (1) the iStar model is extended and normalized into gStar in order to incorporate the system-oriented information required for formalization; on this basis, temporal behavioral obligations are systematically derived and organized as a template-guided set of LTL formulae for specifying safety and progress properties; and (2) the enriched gStar model is automatically transformed into Event-B specifications in a model-driven engineering manner, where the resulting formal artifacts support proof-obligation-based and model-checking-based analysis while preserving traceability between requirements models and behavioral specifications. On top of this process, the paper further develops a verification-oriented formal semantics for the considered gStar fragment, in which core constructs such as goals, operations, obstacles, domain assumptions, refinements, and dependencies are interpreted systematically in the target Event-B domain, thereby providing the semantic basis for the mapping rules and for the formal analysis of the transformed models.

The main contributions of this work are as follows:

- An extension of iStar into gStar to strengthen system-aspect modeling for formalization.
- A verification-oriented formal semantics for the considered gStar fragment, serving as the basis for interpreting core constructs in Event-B.
- A set of mapping rules and an automated model transformation from gStar to Event-B, implemented in ATL, together with an

Acceleo-based generation step for producing textual Event-B artifacts.

- A template-guided mechanism for constructing LTL formulae to specify safety and progress properties, with explicit traceability to source model elements.
- A case study and experimental results illustrating the feasibility of the proposed approach.

The remainder of this paper is organized as follows. Section 2 presents the research motivation and theoretical background. Section 3 outlines the core ideas of the proposed approach. Section 4 details the extensions and normalization steps added to iStar in order to form gStar in a system-oriented manner. Section 5 presents the semantic mapping rules from gStar/iStar to Event-B together with their automated implementation in ATL and the Acceleo-based generation of textual Event-B artifacts. Section 6 introduces the template-guided construction of LTL formulae and their traceability to model elements. Section 7 describes tool support. Section 8 discusses related work. Finally, Section 9 concludes the paper and outlines directions for future work.

2. Motivation and Background

This section motivates our study with an example and provides the necessary background.

2.1. Motivating Example

Modern software systems are increasingly required to satisfy not only functional requirements but also temporal and ordering obligations, especially in settings involving multi-party interactions and dependencies on external services. Meeting scheduling is a representative domain that clearly exhibits these characteristics: multiple actors participate, heterogeneous constraints arise from personal calendars and meeting-room resources, and

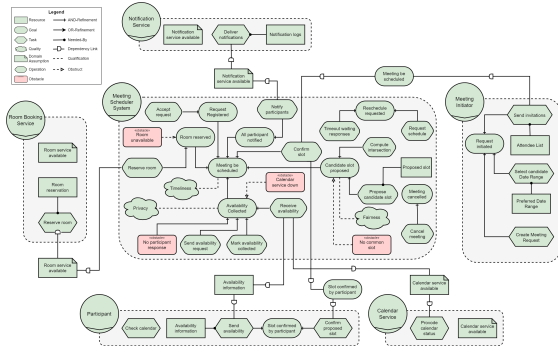


Figure 1. gStar model of meeting scheduling.

infrastructure services may become unavailable during execution. Owing to its representativeness and the richness of actor dependencies, meeting scheduling has long been used in the iStar community as a motivating example for early-phase requirements modeling and goal reasoning in organizational contexts.

Figure 1 presents the motivating example used throughout this paper. The model is an extended version of the classical meeting-scheduling example from early-phase requirements engineering [10]. It is reformulated in gStar to make system-oriented aspects explicit, including milestone-like states, operational tasks, and environment-induced obstacles relevant to formal verification.

Figure 1 involves one central system actor and several interacting environment actors:

- **Meeting Scheduler System:** the system actor that executes the scheduling process from request acceptance to final confirmation.
- **Meeting Initiator:** the actor that creates the meeting request and issues invitations.
- **Participant:** the actor that provides availability information and confirms the proposed slot.
- **Calendar Service:** the environment service that provides calendar status and supports

availability retrieval.

- **Room Booking Service:** the environment service that supports meeting-room reservation.
- **Notification Service:** the environment service that provides notification channels and records notification logs.

Separating system and environment actors clarifies which behavior is controlled by the system and which is influenced by external conditions, which is essential for deriving verifiable specifications.

The high-level objective of the example is for the meeting to be scheduled. In the gStar model, this objective is operationalized by a central behavioral flow within the boundary of the Meeting Scheduler System, coordinated with the Meeting Initiator, Participants, and the environment services Calendar Service, Room Booking Service, and Notification Service. The main flow can be summarized as a sequence of operational tasks and milestone-like states as follows:

- **Request initiation and acceptance:** the process starts at Meeting Initiator with Create Meeting Request, reaching Request initiated. The request is then handled by the system via Accept request, establishing the milestone Request Registered.
- **Availability request and collection:** after the request is registered, the system performs Send availability request. At Participant, Check calendar and Send availability produce Availability information. The system receives it through Receive availability and completes Mark availability collected to reach Availability Collected.
- **Feasibility computation and slot proposal:** from Availability Collected, the system executes Compute intersection. If a non-empty intersection exists, the system

performs Propose candidate slot, reaching Candidate slot proposed and producing Proposed slot.

- **Room reservation:** after a candidate slot is proposed, the system performs Reserve room, interacting with Room Booking Service and the resource Room reservation. When successful, the milestone Room reserved is reached.
- **Participant notification:** once Room reserved holds, the system performs Notify participants via Notification Service. Completion is captured by All participants notified and is associated with Notification logs on the service side.
- **Slot confirmation and completion:** at Participant, Confirm proposed slot leads to Slot confirmed by participant. The system then completes Confirm slot, thereby achieving the objective of scheduling the meeting.

Beyond functional objectives, Figure 1 also implies quality attributes that guide the properties to be verified at the formal level. These quality attributes can be grouped into four categories:

- **Timeliness:** requiring timely progress during availability collection; after Sendavailabilityrequest, the system should reach Receiveavailability within a bounded waiting period, or trigger a timeout transition (e.g., op_sys_timeout) to proceed with subsequent handling such as rescheduling or cancellation.
- **Ordering consistency:** requiring correct ordering of milestones; for instance, All participants notified should hold only when Room reserved holds, and Meeting Scheduled should hold only when Slot confirmed by participant holds.

- **Service availability:** capturing external conditions of Calendar Service and Room Booking Service that directly affect progress, such as ENV_CAL_DOWN, which must be handled to avoid non-progressing executions.
- **Privacy:** requiring that Availability information is processed only within a valid scheduling context, avoiding collection or recording states when the process has not been properly initiated or has already been terminated.

In addition, several key obstacles and corresponding handling branches are incorporated to prevent non-progressing states:

- **No participant response:** after Sendavailabilityrequest, the system may fail to receive sufficient responses; op_sys_timeout sets responseTimeout, followed by op_sys_request_reschedule to reach rescheduleRequested, ensuring that the process does not wait indefinitely.
- **Calendar service down (ENV_CAL_DOWN):** when the calendar service becomes unavailable, availability-collection steps may be disabled; this obstacle represents adverse environment conditions and may lead to non-progressing or terminal situations unless suitable assumptions or handling branches are introduced at the verification level.
- **No common slot (ENV_NO_COMMON_SLOT):** after Computeintersection, the intersection may be empty, preventing Proposecandidateslot; the handling branch triggers op_sys_request_reschedule to reach rescheduleRequested.
- **Room unavailable (ENV_ROOM_UNAVAILABLE):** after a

slot is proposed, Reserve room may fail due to lack of rooms; the system then moves to `rescheduleRequested` or terminates via `op_sys_cancel` to reach cancelled, thereby ensuring controlled termination.

These branches provide the basis for specifying and checking progress properties, non-progress situations, and terminal-state behavior in subsequent verification steps.

2.2. Goal-Oriented Requirements Engineering and iStar

In Goal-Oriented Requirements Engineering (GORE), requirements are modeled in terms of stakeholders' intentions and the dependency relationships among actors. iStar is a widely used GORE framework because it explicitly captures actors, intentional elements, and dependency structures in organizational settings [1].

The iStar standard organizes models into complementary views, notably the Strategic Dependency (SD) view and the Strategic Rationale (SR) view. In the SD view, the model emphasizes actor dependencies following the *dependee–dependum–depender* structure. In the SR view, the model explains the internal rationale of an actor using intentional elements such as *Goal*, *Task*, *Resource*, and *Quality*, together with refinement and contribution relationships. Figure 2 depicts the core iStar metamodel from both SD and SR perspectives, clarifying the foundational concepts inherited by gStar as an extension of iStar.

In addition, iStar defines integrity constraints to ensure model well-formedness. For example, dependencies must be consistent with actor roles, refinements must not introduce cycles, and a dependency must not have the same actor as both depender and dependee. These constraints provide a basis for well-formedness checking at the metamodel and instance levels before automated transformation, thereby reducing the risk of propagating source-model defects into formal specifications.

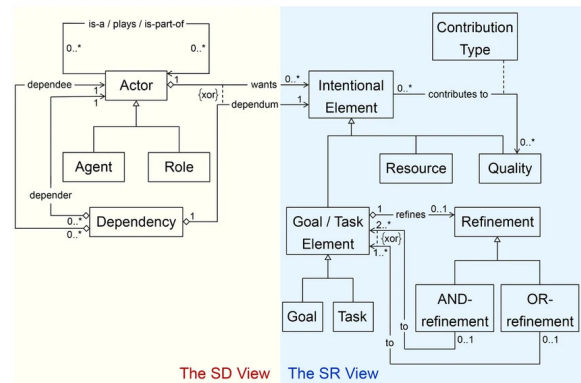


Figure 2. iStar metamodel.

2.3. Event-B: Operational Modeling with Invariants and Events

Event-B is a formal method based on *invariant-based development*, in which safety properties are specified as invariants and system behavior is specified as events. An Event-B model is typically structured into a *context* and a *machine*: the context declares sets, constants, and axioms, whereas the machine declares variables, invariants, and events [11].

In Event-B, each event is defined by *guards* (enabling conditions) and *actions* (state updates). The guard/action structure supports stepwise state transitions while ensuring that invariants are preserved. Figure 3 shows the Event-B metamodel at the EMF level, including the *Project* structure, the *sees* relationship between machine and context, and core entities such as variables, invariants, and events; each event is composed of guards and actions.

2.4. Rodin and ProB for Event-B Verification

Rodin is an integrated environment for Event-B that supports proof-based development by generating and discharging proof obligations (POs) to show that invariants are preserved by events [5]. Complementarily, ProB provides animation and model checking for Event-B, enabling state-space exploration, deadlock detection, invariant-violation detection, and coverage statistics over explored executions [6].

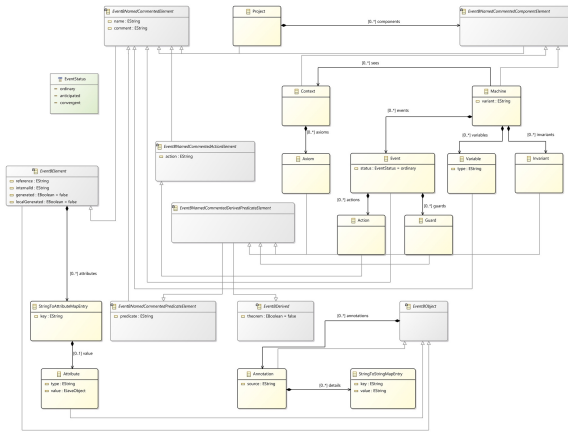


Figure 3. Event-B metamodel.

2.5. Linear Temporal Logic (LTL) for Temporal and Progress Properties

Linear Temporal Logic (LTL) is a linear-time temporal logic for specifying system properties over time, where time is viewed as a sequence of successive states, often assumed to be infinite [9]. LTL is well suited to expressing temporal requirements such as “something must always hold” and “something will eventually happen,” which are not fully captured by invariant-based specifications, especially when reasoning about progress and fairness-like properties.

In LTL, atomic propositions are state predicates evaluated as true or false in each state, for example when a Boolean flag is true or a condition over state variables holds. LTL provides standard temporal operators for relating propositions over time, including *Globally* (G), *Eventually* (F), *Next* (X), and *Until* (U) [12, 13]. Using these operators, LTL commonly specifies (i) *safety* properties (“bad things never happen”), often written as $G \neg \text{bad}$, and (ii) *liveness* or *progress* properties (“good things eventually happen”), often written as $F \text{ good}$ or $G(\text{trigger} \rightarrow F \text{ goal})$.

Within the Event-B ecosystem, ProB supports checking LTL properties formulated over state predicates of Event-B machines during model checking [6]. A common practice is to map atomic

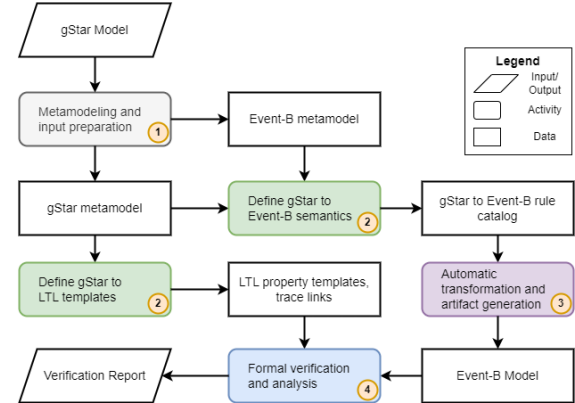


Figure 4. Overview of our approach.

propositions to predicates over Event-B state variables and then verify properties such as the preservation of milestone conditions, the eventual reachability of milestones, or the satisfaction of ordering constraints between milestones along all executions.

3. Overview of Approach

This section outlines an approach that combines formal semantic rule definition and MDE-based automation to transform gStar models (as an extension of iStar) into verifiable Event-B specifications, while also supporting template-guided derivation of temporal properties in LTL. Figure 4 depicts the overall workflow and produced artifacts, starting from the input gStar model, passing through a rule-based semantic mapping layer, then being automated via ATL/Accelleo for Event-B artifact generation, and finally being analyzed and verified using Rodin/ProB.

Step 1: Language modeling and input preparation. The gStar metamodel and the EMF-based Event-B metamodel are first defined. The gStar instance model is then prepared to make the system–environment boundary, operational tasks, milestone-like states, and environment-induced obstacles explicit, so that subsequent semantic

mapping and verification can be performed consistently.

Step 2: Formal semantics as transformation rules. A set of gStar→Event-B mapping rules is defined to translate core gStar concepts into Event-B contexts/machines, variables, invariants, and events (guards/actions). In addition, a set of template-guided derivation rules is introduced to express progress and milestone-ordering obligations as LTL formulae, with explicit trace links to the corresponding Event-B state variables.

Step 3: Tool-supported automation and artifact generation. Based on the above rules, ATL is used to automate the M2M transformation from gStar XMI to Event-B XMI conforming to the target metamodel. Aceleo is then applied to generate textual Event-B artifacts (.buc/.bum) from the resulting Event-B XMI, supporting downstream use within the Rodin/ProB-based verification workflow.

Step 4: Verification and evaluation. The generated Event-B model is verified in Rodin by discharging proof obligations for invariant preservation, and it is further analyzed in ProB via model checking to explore the state space, identify deadlocks, and obtain exploration statistics. The derived LTL formulae serve as a complementary validation layer for progress and ordering properties, while the generated Event-B artifacts are compared against the Event-B oracle to assess transformation correctness and coverage.

4. gStar Metamodel Design

This section presents the design of the gStar metamodel as a goal-directed extension of iStar, aiming to make system-oriented aspects explicit in order to support transformation to verifiable specifications [1].

4.1. Design Rationale and Extension Scope

The goal of gStar is to extend iStar in a system-oriented manner, so that the requirements model

not only captures intentions and dependencies, but also provides sufficient structure to be transformed into an operational specification that can be proved and model-checked. This direction follows the standardization objective of iStar, where the conceptual core is defined consistently to serve as a foundation for subsequent extensions.

The applied extension principles are:

- Preserve the iStar core as much as possible and maintain the SD/SR structuring to preserve the semantics of goals and dependencies.
- Add only the minimum required constructs to model operational behavior, milestone/state information, and environment conditions, directly supporting transformation and verification.
- Strengthen traceability via shared attributes (identifier, name, description) for model elements.

To provide a coherent explanation, the gStar extensions are discussed through four analysis dimensions commonly used in goal-oriented requirements engineering: Why (goals and obstacles), Who (actors and responsibility boundaries), How (operational tasks), and What (domain knowledge and conditions) [14, 15].

4.2. Main Extensions of gStar over the iStar Core

Figure 5 presents the complete gStar metamodel. The inherited iStar core is preserved, while the gStar extension introduces additional classes and attributes to increase behavior-oriented modeling and verification readiness.

First, iStar already supports the Who aspect effectively via actors and dependencies. However, the iStar metamodel itself does not provide a standard mechanism to mark which actor represents the system whose behavior must be specified, and which actors represent external environment services. When moving toward

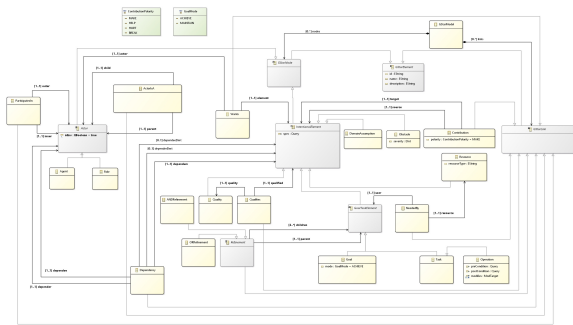


Figure 5. The gStar metamodel.

an operational specification, the lack of this information easily causes ambiguity about the control scope. Therefore, gStar introduces the `isEnv` attribute to classify environment actors within `Actor`, making the system–environment boundary explicit at the source-model level. This addition helps identify the system actor as the central scope for generating the machine, and supports interpreting non-progressing situations or deadlocks caused by environment conditions in later verification steps.

Next, iStar represents goals and refinements well, but verification commonly requires a clear distinction between obligations to achieve and obligations to maintain, in order to select an appropriate specification style. In addition, iStar does not aim to model obstacles as traceable elements to support analyzing goal failures and designing handling branches. Based on obstacle analysis in goal-oriented requirements engineering, gStar introduces goal modes via `GoalMode` and the `Goal.mode` attribute (ACHIEVE/MAINTAIN), and introduces `Obstacle` as an intentional element. With these additions, the *Why* aspect is made explicit in gStar: the model distinguishes goals to be achieved from goals to be maintained, and it represents obstacles together with corresponding handling directions to ensure progress does not fall into indefinite waiting or blocking.

Regarding the *How* aspect, iStar includes `Task` mainly for modeling what needs to be done

at the intentional level, without clearly separating operational state-changing actions executed by the system as part of the process. If every `Task` were directly mapped to an event of an operational specification, the target model could become bloated and difficult to verify. Therefore, gStar specializes `Task` into `Operation` to explicitly represent system operational steps. `Operation` has two attributes, `guard` as a precondition and `action` as the operation’s action, reducing ambiguity between intention-level tasks and operational actions.

Finally, to clarify the *What* aspect of domain knowledge and conditions, conditions must be represented structurally rather than only in natural language. While iStar focuses on goals and dependencies, domain conditions often appear as informal descriptions or scattered annotations, which are difficult to reuse and to process systematically. Therefore, gStar introduces `DomainAssumption` to record domain/environment assumptions, and introduces an expression sublanguage `Expr` to represent logical conditions and relational comparisons in a structured form. The `Expr` hierarchy includes logical operators, relational operators, typed constants, and variable/reference mechanisms (with a prime flag to distinguish pre/post state). As a result, state predicates and domain constraints are represented explicitly, consistently, and can be reused throughout the model [13].

In summary, the gStar metamodel is designed by preserving the iStar core to maintain goal and dependency modeling, while selectively adding constructs that iStar does not target at the operational level: system–environment classification, goal modes and obstacles, operational tasks, domain assumptions and structured expressions, and a traceability-oriented foundation. Based on this metamodel, the next section presents the rule set for transforming gStar to Event-B.

5. Semantic Mapping Rules from gStar to Event-B

This section presents the semantic mapping rules from gStar to Event-B in a rule-based semantic style. The objective is not merely to define a syntactic transformation between two modeling notations, but to establish a verification-oriented formal interpretation for the considered gStar fragment, so that the meaning of its core constructs is systematically reflected in the resulting Event-B model.

In this work, Event-B is used as the target semantic domain for analysis. System state is represented by variables constrained by invariants, while behavior is described by events equipped with guards and actions. This allows the generated model to be analyzed in Rodin through proof obligations and in ProB through animation, model checking, deadlock detection, and temporal verification.

The presentation is organized as follows. We first introduce the principles, scope, and assumptions of the mapping. We then describe the architectural rules used to construct the Event-B skeleton, followed by the semantic interpretation of state-, obligation-, and behavior-related constructs. Next, we define the mapping of expressions, state the preservation claims for the considered fragment, and finally explain how the rule set is mechanized in an executable MDE pipeline using ATL and Acceleo.

5.1. Principles, Scope, and Assumptions

The purpose of the rule set is not to define a complete formal semantics for the whole of gStar/iStar, but rather to provide a verification-oriented formal semantics for the fragment of constructs that can be meaningfully analyzed in Event-B. Accordingly, only those gStar elements that directly contribute to system state, behavioral evolution, constraints, or verification-relevant environmental conditions are mapped into the target formal model. Constructs outside

this fragment may still remain useful at the requirements level, but they are not necessarily encoded directly in the Event-B specification.

In this work, the target semantic domain is the standard Event-B structure composed of a *Context* and a *Machine*. The *Context* represents the static part of the problem domain, including carrier sets, constants, and foundational domain assumptions. The *Machine* represents the dynamic part, including state variables, invariants, and events. Under this interpretation, the meaning of a gStar model is not treated merely as an organizational requirements diagram, but as a verification-oriented behavioral specification that can be analyzed through proof and state-space exploration.

The scope of the transformation focuses primarily on the behavior of the system actor. The environment-actor classification attribute in gStar is used to distinguish those parts of the model that are under system control from those that belong to the environment. Accordingly, the machine is mainly generated from the system actor, while environment-related aspects are reflected indirectly through domain assumptions, environment variables, guards, or explicit environment events when needed for progress, deadlock, and temporal analysis.

At the implementation level, this semantic mapping is realized as a two-stage MDE pipeline. First, ATL is used to transform a gStar XMI model into an Event-B XMI model conforming to the target metamodel. Second, Acceleo is used to generate textual Event-B artifacts (.buc/.bum) from the resulting Event-B XMI. Hence, the mapping is not only defined at the semantic-rule level but also operationalized as an executable transformation workflow.

5.2. Architectural Rules for Constructing the Target Model

The first rule group determines the architectural skeleton of the target Event-B model. These rules answer the question of

Table 1. Structural mapping rules.

Rule	Source (gStar)	Target (Event-B)
R-S1	gStar model	Project
R-S2	Domain vocabulary and static assumptions	Context (sets/constants/axioms)
R-S3	System actor	Machine
R-S4	Domain usage relation	Machine sees Context

how a gStar model, when viewed through a verification-oriented lens, should be reorganized in the conceptual space of Event-B.

These architectural rules do not directly encode business behavior, but they establish the formal space in which state meanings and behavioral meanings are subsequently placed. This is necessary because gStar is a high-level requirements model, whereas Event-B requires a formal structure that clearly separates static and dynamic aspects.

Under this interpretation, the *Project* acts only as a container, whereas the actual formal meaning is carried by the *Context–Machine* pair. The *Context* contains elements assumed to remain fixed during analysis, such as carrier sets, domain constants, and foundational axioms. The *Machine* contains the state that may evolve, together with the rules governing that evolution. This separation follows the standard Event-B organization and reflects the needs of verification: what belongs to the domain basis is placed in the context, and what belongs to system evolution is placed in the machine.

For the MeetingScheduler case study, a representative fragment of the generated context is as follows:

```
sets B
constants tt, ff, MAX_TRIES
axioms
partition(B, {tt}, {ff})
tt ≠ ff
```

$$MAX_TRIES \in \mathbb{N}_1$$

$$MAX_TRIES = 3$$

This fragment makes explicit the static typing basis and domain constants used by the generated machine.

5.3. Semantic Interpretation of Core gStar Constructs

In the considered fragment, system actors are interpreted as the primary source of controllable behavior and are therefore encoded mainly in the Event-B machine. Environment actors are treated as exogenous influences and are reflected through assumptions, guards, environment variables, or dedicated environment events when needed. Maintain goals are interpreted as state properties that must hold in all reachable states and are therefore mapped to invariants, whereas achieve goals and milestone-like goals are represented through state variables and state predicates so that progress can be analyzed. Operations are interpreted as executable transitions and are mapped to events. Domain assumptions are encoded as axioms, invariants, or guards depending on whether they constrain the static or dynamic part of the model. Obstacles are interpreted as hindering conditions or sources of exceptional evolution, while refinements and dependencies are preserved through constraints over invariants, guards, event structures, or temporal obligations.

Maintain goals. In this work, a maintain goal is interpreted as a state property that must be preserved throughout execution. It is therefore encoded as an Event-B invariant. This is the most natural semantic mapping, since invariants in Event-B precisely capture properties that must remain true in all reachable states.

For example, if the source model includes a safety requirement stating that the system must never be both scheduled and cancelled, the corresponding Event-B encoding takes the form of an invariant such as:

$$scheduled = tt \Rightarrow cancelled = ff$$

More generally, ordering constraints such as

$$\begin{aligned} scheduled &= tt \Rightarrow notified = tt \\ notified &= tt \Rightarrow roomReserved = tt \\ roomReserved &= tt \Rightarrow slotProposed = tt \end{aligned}$$

are also interpreted as maintain-like obligations over process consistency.

Achieve goals and milestone-like states. A verification-oriented semantics must also clarify how achieve goals are treated. In this work, achieve goals and milestone-like goals are interpreted as state milestones that should become reachable during execution. Such meanings are not naturally represented by invariants, since invariants describe what must always hold rather than what should eventually be reached. Accordingly, achieve goals are reflected through state variables, milestone flags, or corresponding state predicates updated by events.

For the MeetingScheduler case study, milestones such as requestReceived, availCollected, slotProposed, roomReserved, notified, scheduled, rescheduleRequested, and cancelled are represented as Boolean state variables or equivalent state conditions. This representation supports subsequent analysis of progress and response properties through LTL and state-space exploration.

Refinement. AND/OR refinement is not treated merely as a graphical relation between source elements, but as a constraint on the satisfaction structure of the model. In the AND case, operational realization at a lower level must reflect conjunction or coordinated satisfaction of multiple subconditions or substeps. In the OR case, the target model must preserve the existence of alternative satisfaction paths.

Since Event-B does not directly provide the same intentional refinement structure as gStar/iStar, refinement is encoded indirectly

through combinations of invariants, guards, and event-enabling conditions. Thus, refinement is preserved in a verification-oriented sense: the target model still reflects the logic of conjunctive or alternative decomposition, even if it does not reproduce the original graphical form.

Domain assumptions. A *DomainAssumption* is interpreted as a domain condition that constrains analysis or enables certain behaviors. Depending on its nature, it is encoded either as an axiom in the *Context* or as an invariant or guard in the *Machine*.

If the assumption is static and foundational, such as a typing restriction or fixed domain relation, it is placed in the *Context*. If it directly affects the executability of system behavior, it is reflected through a machine invariant or an event guard. This distinction avoids leaving assumptions implicit and makes the basis of proof and counterexample analysis explicit.

Obstacles. An *Obstacle* is interpreted as a condition that may block nominal progress, divert the execution path, or force the system into an exceptional-handling branch. Under this view, an obstacle is not a merely descriptive annotation in the requirements model, but a source of non-nominal behavior that must remain analyzable in the target model.

In Event-B, an obstacle may be reflected as a negated guard for the nominal path, as an enabling condition for an exceptional branch, as a dedicated state condition, or as a handling event for situations such as timeout, rescheduling, cancellation, or room unavailability. This allows the target model to support deadlock analysis, non-progress analysis, and temporal verification.

Dependencies. Dependencies are interpreted as cross-actor or cross-element relations that constrain satisfaction or execution. In the considered fragment, these relations are preserved not by reproducing the original dependency graph directly, but by encoding their operational consequences in the target model. Depending on the nature of the dependency, this may take the form of state relations, invariants, guards, or

temporal obligations checked after transformation. In this sense, dependencies are preserved at the verification level rather than at the graphical level.

A representative fragment of the generated machine contains typed state invariants such as:

```

scheduled ∈ B
cancelled ∈ B
notified ∈ B
roomReserved ∈ B
tries ∈ ℕ
tries ≤ MAX_TRIES

```

together with ordering and safety constraints such as:

```

scheduled = tt ⇒ notified = tt
notified = tt ⇒ roomReserved = tt
roomReserved = tt ⇒ slotProposed = tt
scheduled = tt ⇒ cancelled = ff

```

These invariants illustrate how milestone typing, process ordering, and basic safety obligations are preserved in the target Event-B model.

5.4. Semantic Mapping Rules for Behavior

Behavioral semantics is centered on the interpretation of operations as transitions. In the considered fragment, an *Operation* is interpreted as a basic behavioral unit of the system actor, namely a step with an enabling condition and a state-update effect. For this reason, it is encoded directly as an Event-B event. This is not only structurally convenient, but semantically appropriate: an operation in gStar denotes doing something under certain conditions, whereas an event in Event-B denotes a transition enabled when its guards hold.

The guard of an operation is interpreted as the condition under which the corresponding behavior may occur. It is therefore mapped to the guards of the target event. Preserving guards is essential for preserving execution meaning: if a source operation is permitted only when certain preconditions hold, then the generated

event should also be enabled only under those same conditions.

Similarly, the action of an operation is interpreted as the effect that updates the system state when the operation is executed. It is therefore mapped to the actions of the target event. The intended meaning is not merely that something is updated, but that the relevant milestone or state variables are updated according to the logic of the original operational step.

A representative example is the generated event corresponding to the notification step:

```

event op_sys_notify
when
  roomReserved = tt
  cancelled = ff
  notifyConnected = tt
then
  notified := tt
end

```

In this example, successful room reservation and non-cancellation act as process preconditions, while *notifyConnected* = *tt* reflects an environment-related enabling condition. The action *notified* := *tt* records completion of the corresponding milestone. The generated event thus preserves the operational meaning of the source task.

Obstacle-handling branches are interpreted as exceptional or recovery behavior when nominal progress cannot continue. They are therefore encoded as one or more dedicated handling events. This is crucial for ensuring that the target model does not represent only the nominal path, but also includes timeout, no-common-slot, room-unavailable, or cancellation situations.

By means of these handling events, the target model can explicitly represent adverse situations and can support temporal analysis of properties such as the eventual triggering of rescheduling after timeout, the eventual request for rescheduling when no common slot exists, or the eventual completion of a request through scheduling or

Table 2. Expression mapping rules.

Rule	Source (gStar)	Target (Event-B)
R-E1	And(e1,e2)	$\llbracket e1 \rrbracket \wedge \llbracket e2 \rrbracket$
R-E2	Or(e1,e2)	$\llbracket e1 \rrbracket \vee \llbracket e2 \rrbracket$
R-E3	Implies(e1,e2)	$\llbracket e1 \rrbracket \Rightarrow \llbracket e2 \rrbracket$
R-E4	Not(e)	$\neg \llbracket e \rrbracket$
R-E5	RelOp (op,e1,e2)	$\llbracket e1 \rrbracket \text{ op } \llbracket e2 \rrbracket$ $op \in \{=, \neq, <, \leq, >, \geq\}$
R-E6	Const (type,literal)	literal value (Boolean / Integer / String)
R-E7	VarRef(v, prime=false)	v
R-E8	VarRef(v, prime=true)	post-state value of v in actions
R-E9	AttrRef (v, attr, ...)	attribute reference according to the mapping convention

cancellation.

5.5. Expression Mapping Rules

To complete the semantics of guards and actions, the gStar expression sublanguage must also be interpreted in Event-B. In this work, the interpretation function is denoted by $\llbracket \cdot \rrbracket$, mapping gStar expressions into Event-B predicates or update expressions.

These rules ensure that logical conditions and update expressions in the source model are preserved consistently in the target model. Logical and relational operators retain their predicate-level meaning, while typed constants remain explicit to avoid ambiguity in generated conditions. The convention `prime=true` is necessary for distinguishing pre-state and post-state values, especially when an operation describes a state update.

In the MeetingScheduler case study, expressions stating that later steps are only allowed after earlier milestones have been reached, or that a notification is only possible when a communication service is available, are translated into guards by means of $\llbracket \cdot \rrbracket$. Similarly,

updates of milestone flags and counting variables are translated into actions according to the pre-state/post-state convention.

5.6. Application to the MeetingScheduler Case Study

Applied to the MeetingScheduler case study, the above rules yield a concrete Event-B model structured as one project, one context, and one machine. The generated context captures the static part of the domain, including Boolean typing, counting constants, and foundational domain assumptions. The generated machine captures the dynamic part of the scheduling process through state variables, invariants, and events.

Milestones such as request registration, availability collection, completion of availability gathering, slot proposal, room reservation, notification, confirmation, rescheduling, cancellation, and successful scheduling are reflected as state variables or state conditions in the machine. Safety and ordering requirements are reflected in invariants. Operations such as accepting a request, receiving availability, marking availability as collected, computing intersections, proposing a slot, reserving a room, notifying participants, confirming the slot, requesting rescheduling, and cancelling the process are translated into events with corresponding guards and actions. Adverse situations such as timeout, lack of a common slot, or room unavailability are reflected by handling events or related enabling conditions.

Therefore, the generated Event-B model is not merely a structural translation, but a verification-oriented formal representation of the MeetingScheduler process.

5.7. Semantic Preservation Statements

To elevate the above rules from the level of transformation correspondences toward a verification-oriented formal semantics, we now state preservation properties for the considered fragment. These statements do not claim a

complete semantics for all of gStar, but rather clarify that, for the constructs covered by the rule set, the mapping preserves the intended verification-relevant meaning.

Proposition 1 (Preservation of maintain-goal meaning). For every maintain goal g translated into an invariant I_g in the target machine, if the invariant-preservation proof obligations of the generated events are discharged, then I_g is preserved over all reachable states of the generated Event-B model.

Interpretation. This proposition states that the meaning of a maintain goal as something that must always hold is preserved in the target domain through invariant preservation.

Proposition 2 (Operational correspondence of operations). For every operation op translated into an Event-B event e , the guards of e represent the enabling conditions of op , and the actions of e represent the principal state-update effect of op .

Interpretation. This proposition states that an operation does not become a semantically empty syntactic label in the target model, but retains its role as a conditional state-transition step.

Proposition 3 (Correspondence of obstacle handling). For every obstacle or obstacle-handling branch included in the mapping scope, the generated Event-B model contains a corresponding hindering condition, exceptional branch, or handling event reflecting the deviation from the nominal process path.

Interpretation. This proposition states that obstacle-related meaning is not lost in the transformation, but preserved in a form analyzable by deadlock checking, invariant checking, and temporal verification.

Proposition 4 (Assumption-aware preservation of domain assumptions and environment-dependent behavior). For every domain assumption translated into a context axiom, machine invariant, or event guard, proof-based and model-checking results over the target model are

to be interpreted under the explicitly represented assumptions.

Interpretation. This proposition is essential for explaining why certain progress properties must be read in an assumption–guarantee style, especially for behaviors depending on the environment such as participant confirmation.

Proposition 5 (Preservation of refinement structure at the verification level). AND/OR refinement relations are translated into conjunctive or alternative constraints over invariants, guards, and event structures such that the source logic of joint necessity or alternative satisfaction remains reflected in the target model.

Interpretation. This proposition does not claim preservation of every intentional nuance of refinement in iStar/gStar, but it does claim preservation of the decomposition structure relevant to verification.

5.8. Executable Realization of the Mapping

The above rule set is realized as a concrete transformation workflow. The gStar-to-Event-B model transformation is mechanized in ATL, producing an Event-B XMI model containing the generated project, context, machine, variables, invariants, events, guards, and actions. This intermediate model is then processed by Acceleo in order to generate textual Event-B artifacts in .buc and .bum form.

Therefore, the proposed mapping does not remain at the level of declarative rules. It is executable within an MDE pipeline and yields Event-B models that can be further analyzed in Rodin and ProB. In this sense, the present section provides a verification-oriented formal semantics for the considered gStar fragment, while also supplying the operational bridge between the requirements model and the target formal model.

6. LTL Property Templates Derived from gStar

This section describes how temporal properties are specified in Linear Temporal Logic to support complementary verification of the operational model. The approach follows a template-guided, rule-based style: it first defines how gStar elements are anchored to observable state propositions in the verification model, and then instantiates corresponding LTL templates for refinement relations, milestone ordering, obstacle handling, and dependencies among actors.

6.1. Template-Guided Rule Set for Deriving LTL from gStar

Table 3 summarizes the template-guided rule set. **R0** provides the basis for constructing $\text{Sat}(X)$ from a model element; the remaining rules define templates for refinement consistency, progress, milestone ordering, obstacle handling, and dependency. In addition, a set of safety domain constraints is used to exclude contradictory states. The LTL formulae follow the ProB syntax, where state propositions must be enclosed in braces. The notation $\text{Sat}(X)$ denotes the state proposition corresponding to element X in the gStar model; in the ProB syntax, $\text{Sat}(X)$ is realized as a state predicate $\{P_X\}$. The temporal operators used in this section are G (globally) and F (eventually).

R0 requires defining a concrete state predicate P_X for each element X . In the case study, P_X is taken from state flags or state conditions already present in the operational model. **R1** is used when an achieve-type goal is formulated as a conditional response: when the trigger holds, the goal must eventually hold. **R3** preserves refinement consistency along the parent–child relation. **R5** encodes the obligation to progress to the next milestone, preventing indefinite stuttering at an intermediate milestone. **R7** requires that when an obstacle occurs, the handling branch is eventually reached. **R10** captures dependencies in a request–response form. **R11** is used to

Table 3. Template-guided rule set for deriving LTL formulae from gStar.

Rule	Applies to	Template formula
R0	Specification / Expression X	$\text{Sat}(X) = \{P_X\}$
R1	Achieve-type goal with trigger $\text{Tr}(G)$	$G(\text{Tr}(G) \Rightarrow F \text{Sat}(G))$
R3	AND-refinement $P = \text{AND}(C1..Cn)$	$G(\text{Sat}(P) \Rightarrow (\text{Sat}(C1) \wedge \dots \wedge \text{Sat}(Cn)))$
R5	Ordering / Chain $C1 < C2 < \dots < Cn$	For all i : $G(\text{Sat}(Ci) \Rightarrow F \text{Sat}(Ci + 1))$
R7	Obstacle O with handling $M(O)$	$G(\text{Sat}(O) \Rightarrow F \text{Sat}(M(O)))$
R10	Dependency D (request / provide)	$G(\text{Request}(D) \Rightarrow F \text{Provide}(D))$
R11	Assumption–guarantee reporting	A from assumptions, G from guarantees

report results in an assumption–guarantee style, where assumptions typically originate from the environment or the dependee side.

6.2. Instantiated LTL Formulae, Traceability, and Interpretation

This subsection presents the instantiated LTL formulae for the MeetingScheduler model, based on the template-guided rule set in Table 3. In the current study, these formulae are instantiated and validated manually, while preserving explicit traceability to the corresponding gStar and Event-B elements. Table 4 summarizes the instantiated formulae, their traceability links, and the rules applied to each formula.

LTL-1 is a minimal safety constraint that excludes contradictory terminal outcomes and ensures consistency of the process result; it requires that there is no state in which the system is simultaneously in the scheduled state and in the cancelled state. **LTL-2** ensures that if the scheduled state is reached then the notified

Table 4. Summary of instantiated LTL formulae, traceability, and applied rules.

ID	LTL property	gStar source elements	Applied rules
LTL-1	$\mathbf{G} \neg(\{scheduled = tt\} \wedge \{cancelled = tt\})$	Goal G_MEETING_SCHEDULED_SYS + Goal G_CANCELLED_SYS	R0 + safety domain rule
LTL-2	$\mathbf{G}(\{scheduled = tt\} \Rightarrow \{notified = tt\})$	ANDRefinement RF_SYS_AND_1 + Operation OP_SYS_NOTIFY + OP_SYS_CONFIRM_SLOT	R0 + R3
LTL-3	$\mathbf{G}(\{intersectionNonEmpty = ff\} \Rightarrow \mathbf{F}\{rescheduleRequested = tt\})$	Obstacle OB_NO_COMMON_SLOT + Obstructs OBS_2 (obstructs G_SLOT_PROPOSED_SYS) + Goal G_RESCHEDULE_REQUESTED_SYS	R0 + R7
LTL-4	$\mathbf{G}(\{responseTimeout = tt\} \Rightarrow \mathbf{F}\{rescheduleRequested = tt\})$	Obstacle OB_NO_RESPONSE + Obstructs OBS_1 (obstructs G_AVAIL_COLLECTED_SYS) + Goal G_RESCHEDULE_REQUESTED_SYS	R0 + R7
LTL-5 (A1)	$\mathbf{G}(\{notified = tt\} \Rightarrow \mathbf{F}\{participantConfirmed = tt\})$	Dependency DP_6 + Task T_PART_CONFIRM_SLOT / Goal G_PART_CONFIRMED	R0 + R10
LTL-6 (G1)	$\mathbf{G}(\{requestReceived = tt\} \Rightarrow \mathbf{F}(\{scheduled = tt\} \vee \{cancelled = tt\}))$	Goal G_REQUEST_REGISTERED_SYS + Goal G_MEETING_SCHEDULED_SYS + Goal G_CANCELLED_SYS; ANDRefinement RF_SYS_AND_1 and RF_SYS_AND_7	R0 + R1 + R11
LTL-7	$\mathbf{G}(\{availReceived = tt\} \Rightarrow \mathbf{F}\{availCollected = tt\})$	ANDRefinement RF_SYS_AND_3 + Operations OP_SYS_RECEIVE_AVAIL and OP_SYS_MARK_AVAIL_COLLECTED	R0 + R5

milestone must have been reached earlier. Next, **LTL-3** states that when the obstacle of having no common slot occurs, the process must not remain idle; instead, it must eventually reach the reschedule requested state. This is a progress obligation under adverse conditions, ensuring that the model provides a valid handling branch rather than getting stuck at the intersection-computation stage. **LTL-4** requires that when a timeout occurs due to missing responses, the process must eventually branch to rescheduling; this excludes indefinite waiting and ensures an exit mechanism from the no-response situation. **LTL-5** models an assumption about the participant actor's response: after the system has notified, the participant will eventually confirm. **LTL-6** states a process-level response obligation: once a request has been received, the system must eventually reach a valid termination outcome, either scheduled or cancelled. Finally, **LTL-**

7 requires that after the availability received milestone is reached, the process must eventually reach the availability collected milestone; this is a local progress property that helps detect intermediate-state stalling in the availability-collection flow.

7. Tool Support and Experiment

This section presents the tool-based realization of the proposed approach and reports the experimental results obtained from the transformation pipeline. The objective is to show that the mapping rules from gStar to Event-B are not only defined at the conceptual level, but are also mechanized as an executable MDE pipeline that produces Event-B artifacts and enables their subsequent analysis in Rodin and ProB.

7.1. Tool Support

The proposed approach is implemented on top of the Eclipse/EMF ecosystem. At the metamodel level, `gstar.ecore` is used as the source metamodel to represent gStar elements, while `eventb.ecore` is used as the target metamodel to represent the structure of Event-B models in EMF. On this basis, the overall pipeline is organized into three layers: modeling, transformation, and analysis.

At the modeling layer, both the source metamodel and the target metamodel are explicitly specified in Ecore. This provides a precise structural basis for the input and output artifacts throughout the pipeline and ensures that subsequent transformations are performed over clearly defined model structures.

At the transformation layer, the model-to-model phase is implemented in ATL. A gStar model encoded as XMI is transformed into an intermediate Event-B model, also represented in XMI and conforming to `eventb.ecore`. The model-to-text phase is then implemented in Acceleo in order to generate textual Event-B artifacts corresponding to the context and machine components under the `.buc` and `.bum` formats.

At the analysis layer, the generated artifacts are further processed within the Event-B tool ecosystem. In particular, Rodin is used to generate and discharge proof obligations, whereas ProB is used for animation, model checking, deadlock detection, invariant checking, and LTL verification. In this way, the proposed pipeline does not stop at model transformation, but connects directly to formal verification activities.

Figure 6 illustrates the tool pipeline of the proposed approach. As shown in the figure, the input gStar model is represented as XMI conforming to `gstar.ecore`, transformed into an intermediate Event-B model by ATL, and then rendered into textual `.buc` and `.bum` artifacts by Acceleo. These generated artifacts are finally imported into Rodin and ProB for formal analysis.

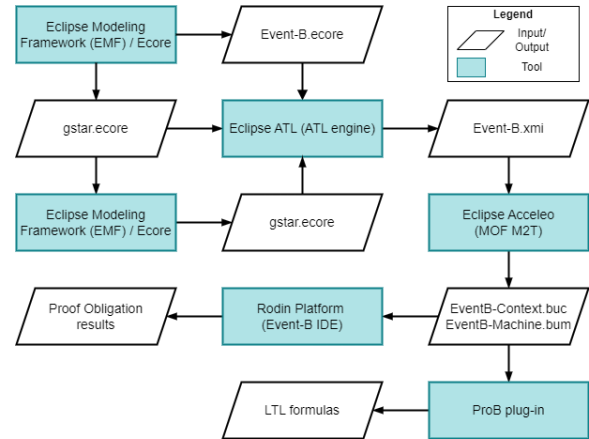


Figure 6. Tool pipeline of the proposed approach.

This organization provides a coherent flow from modeling to transformation and verification.

7.2. Generated Event-B Artifacts

When the pipeline is executed, the ATL transformation produces an Event-B model in XMI form containing the core elements of the target specification, including Project, Context, Machine, as well as the corresponding variables, invariants, events, guards, and actions. This result shows that the proposed mapping rules have been successfully mechanized at the intermediate-model level.

At the structural level, the generated model preserves the standard separation in Event-B between the static and dynamic parts of the specification. The Context is used to represent carrier sets, constants, and selected domain assumptions, whereas the Machine captures state variables, invariants, and events describing the behavior of the process.

At the content level, important state milestones from the source model are reflected as state variables or state predicates in the machine. Safety constraints and execution ordering conditions are encoded through invariants and guards. The operations in gStar, together with obstacle-handling branches, are translated into Event-B events with corresponding enabling conditions and

actions. This indicates that the generated model captures not only the state structure of the source model but also its main behavioral steps.

From the intermediate Event-B model, Acceleo further generates the textual .buc and .bum artifacts, which can be directly imported into Event-B tools. Therefore, the proposed pipeline is realized at both levels: the intermediate-model level and the textual-artifact level for downstream verification.

The generated Event-B artifacts follow the standard Event-B organization into context and machine components, and they can be directly imported into the Rodin platform for subsequent proof and model-checking activities. This confirms that the transformation output is not merely theoretical, but is available as executable artifacts that can be processed by the Event-B toolchain.

7.3. Analysis with Rodin, ProB, and LTL

Proof Obligations in Rodin.

After the Event-B artifacts are generated, the model is imported into Rodin in order to generate and discharge proof obligations. The obtained results show that the proof obligations related to invariant preservation (INV) are successfully discharged for the examined parts of the model. In particular, the obligations associated with INITIALISATION are discharged successfully. In addition, the main process events such as `op_sys_accept_req`, `op_sys_send_avail_request`, `op_sys_receive_avail`, `op_sys_mark_avail_collected`, `op_sys_propose_slot`, `op_sys_reserve_room`, `op_sys_notify`, `op_sys_confirm_slot`, `op_sys_timeout`, and `op_sys_cancel` also preserve their corresponding invariants.

Figure 7 shows the proof-obligation results in Rodin. As can be observed, the INV obligations of INITIALISATION and of the main behavioral events are discharged successfully. This indicates that the Event-B model generated

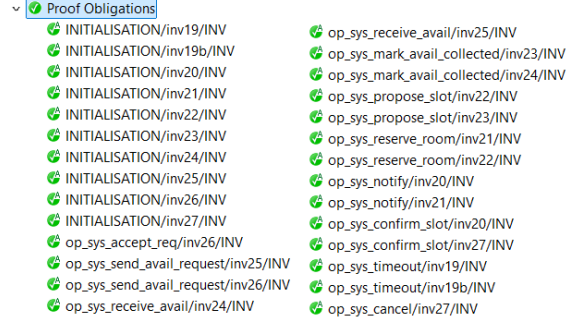


Figure 7. Rodin proof obligations for invariant preservation.

from gStar is consistent with respect to the declared state constraints, in the sense that the principal transitions of the process do not violate the invariants of the machine.

These results also show that the transformation rules from gStar to Event-B produce a target model that can be handled robustly by Rodin. In other words, the output of the pipeline is not only syntactically valid, but also sufficiently well-formed to support proof-based verification through proof obligations.

It should nevertheless be noted that proof obligations in Rodin primarily establish correctness in terms of state-condition preservation. They do not directly address progress, response, or completion properties over time. For this reason, after the proof-obligation analysis, the model is further examined in ProB through model checking and LTL verification in order to assess its dynamic behavioral properties.

Model Checking in ProB.

To analyze the behavior of the model after the Rodin verification step, we use ProB with two checking modes enabled simultaneously: deadlock detection and invariant-violation detection. The result shows that ProB detects a deadlock, meaning that there exists a state in which all event guards evaluate to false and no further transition can be taken. However, within the explored part of the state space, ProB does not report any invariant-violating state.

More precisely, this run yields 15 states and 25 transitions. Among them, the number of invariant-violating states is 0, the number of live states is 8, the number of deadlocked states is 1, the number of open states is 6, and the number of states for which invariants are not yet checked is 6. With respect to coverage, the explored state space includes the events `SETUP_CONSTANTS`, `INITIALISATION`, `op_sys_accept_req`, `op_sys_send_avail_request`, `op_sys_request_reschedule`, `op_sys_cancel`, and `ENV_CAL_DOWN`. By contrast, the events `op_sys_receive_avail`, `op_sys_mark_avail_collected`, `op_sys_compute_int`, `op_sys_propose_slot`, `op_sys_reserve_room`, `op_sys_notify`, `op_sys_confirm_slot`, `op_sys_timeout`, `ENV_PARTICIPANT_CONFIRM`, `ENV_NO_COMMON_SLOT`, and `ENV_ROOM_UNAVAILABLE` remain uncovered in this run.

Figure 8 presents the ProB result for deadlock and invariant checking. As shown in the figure, ProB detects a deadlock while reporting no invariant-violating state. At the same time, the coverage statistics indicate that several events from later stages of the workflow are not triggered in the explored state space. This means that the model still respects the declared state constraints, but it also admits execution branches that terminate early, before the process reaches subsequent stages.

These observations complement the Rodin proof-obligation results in two ways. First, the absence of invariant-violating states reinforces the conclusion that the model preserves its declared state constraints in the explored state space. Second, the presence of a deadlock shows that invariant preservation alone is not sufficient to guarantee that every execution branch continues to progress. In other words, the current model still permits executions that stop before the workflow is completed.

The fact that many events remain uncovered

```
Deadlock found.
ProB has detected a state where all guards are false.
=====
Coverage statistics:
Total Number of States:15
Total Number of Transitions:25
Node Statistics:
invariant_violated:0
live:8
deadlocked:1
open:6
invariant_not_checked:6
total:15
Operations Statistics:
SETUP_CONSTANTS:1
INITIALISATION:1
op_sys_send_avail_request:2
op_sys_cancel:3
op_sys_accept_req:6
op_sys_request_reschedule:6
ENV_CAL_DOWN:6
Uncovered Operations:
op_sys_receive_avail
op_sys_mark_avail_collected
op_sys_compute_int
op_sys_propose_slot
op_sys_reserve_room
op_sys_notify
op_sys_confirm_slot
op_sys_timeout
ENV_PARTICIPANT_CONFIRM
ENV_NO_COMMON_SLOT
ENV_ROOM_UNAVAILABLE
```

Figure 8. ProB result for deadlock and invariant checking.

further clarifies the nature of the detected deadlock. In particular, the steps related to receiving availabilities, marking availability collection as complete, computing intersections, proposing a slot, reserving a room, sending notifications, confirming participation, and handling timeouts are not activated in the explored run. This suggests that some executions terminate prematurely before the process advances to deeper behavioral stages. Such a result directly motivates the need for temporal verification through LTL.

LTL Verification Results.

In addition to proof obligations and state-based model checking, we use ProB to verify a set of LTL formulae representing two main categories of properties: safety properties and progress/response properties. The purpose of this step is to evaluate not only whether the model avoids undesired states, but also whether it actually drives the process toward the intended states over time.

The first result shows that the safety property excluding contradictory final outcomes is satisfied. More specifically, the formula stating that $\text{scheduled} = \text{tt}$ and $\text{cancelled} = \text{tt}$ must never hold simultaneously is confirmed by ProB. This means that, in the checked executions, the model does not allow the system to reach a state in which the request is both successfully scheduled and cancelled at the same time. This result is consistent with the machine invariants and with the proof obligations discharged in Rodin.

ProB also confirms the safety formula $G(\text{not}(\{\text{scheduled} = \text{tt} \ \& \ \text{cancelled} = \text{tt}\}))$ on the analyzed executions. This provides direct evidence that the model correctly encodes the mutual exclusion between the two contradictory outcomes of the process.

By contrast, the remaining formulae in the progress/response category all yield counterexamples in ProB. These counterexamples show that the current model still admits executions in which a triggering condition is reached, but the expected future state is never attained.

First, the formula stating that $\text{availReceived} = \text{tt}$ must eventually lead to $\text{availCollected} = \text{tt}$ is violated. This result shows that the model does not yet guarantee completion of the availability-collection phase. In other words, the model still permits execution branches in which availability information starts being received, but the process never reaches the state indicating that availability collection has been completed. The specific limitation exposed here is the absence of a completion guarantee for an intermediate phase of the workflow.

Second, the formula stating that $\text{requestReceived} = \text{tt}$ must eventually lead to either $\text{scheduled} = \text{tt}$ or $\text{cancelled} = \text{tt}$ is also violated. This is a particularly important result, because it shows that the model does not yet guarantee end-to-end completion for a received request. Once a request enters the system, there still exist executions in which it reaches neither a successful scheduling outcome nor a

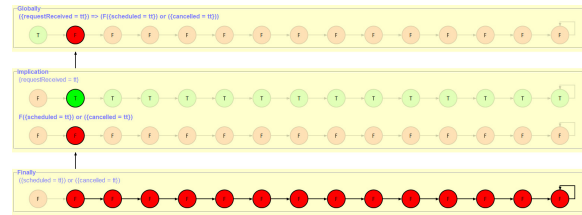


Figure 9. Counterexample for end-to-end request termination.

cancellation outcome. The limitation exposed here is the lack of a termination or resolution guarantee for the overall request-handling process.

Figure 9 illustrates the counterexample for the formula $G(\{\text{requestReceived} = \text{tt}\} \Rightarrow F(\{\text{scheduled} = \text{tt}\} \text{ or } \{\text{cancelled} = \text{tt}\}))$. The figure shows that, after the state requestReceived has been reached, ProB still finds an execution in which the process reaches neither scheduled nor cancelled . This makes the end-to-end progress gap of the current model explicit.

Third, the formula stating that $\text{intersectionNonEmpty} = \text{ff}$ must eventually lead to $\text{rescheduleRequested} = \text{tt}$ is violated. This indicates that the model does not yet enforce the recovery step after detecting that no common time slot exists. From a business-process perspective, once no common slot is available, the system is expected to request rescheduling. However, the ProB counterexample shows that the current model still permits executions in which this adverse condition occurs without triggering the rescheduling behavior. The limitation exposed here is the lack of a guaranteed recovery step when no common availability exists.

Similarly, the formula stating that $\text{responseTimeout} = \text{tt}$ must eventually lead to $\text{rescheduleRequested} = \text{tt}$ is also violated. This result shows that the model does not yet guarantee a recovery mechanism after timeout. In other words, timeouts can occur in the model without forcing the process to continue with a rescheduling request. The limitation exposed here

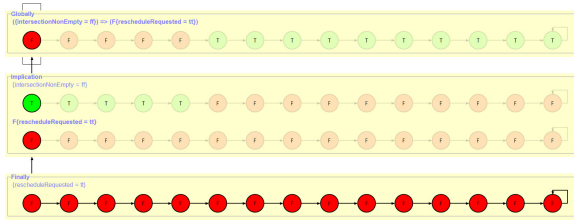


Figure 10. Counterexample for timeout-triggered rescheduling.

is the absence of a recovery guarantee after the response period has expired.

Figure 10 presents the counterexample for the formula $G(\{responseTimeout = tt\} \Rightarrow F(\{rescheduleRequested = tt\}))$. As shown in the figure, ProB finds an execution in which a timeout occurs but the state `rescheduleRequested` is never reached. This clearly indicates that the current model does not yet enforce the expected recovery behavior after timeout.

Finally, the formula stating that `notified = tt` must eventually lead to `participantConfirmed = tt` is violated as well. By nature, this property is strongly environment-dependent, since participant confirmation is not entirely under system control. Even so, the counterexample reveals an important incompleteness in the current model: once a notification has been sent, the model does not yet include sufficiently strong environment assumptions or constraints to ensure that the process eventually reaches confirmation. The limitation exposed here is the lack of a guaranteed environmental response after notification.

Taken together, the LTL counterexamples show that the progress limitations of the current model are concentrated around three points. First, the model does not yet guarantee completion of certain intermediate phases, most notably the availability-collection phase. Second, it does not yet enforce recovery steps after adverse conditions such as the absence of a common slot or the expiration of the response period. Third, it does

not yet guarantee end-to-end completion of the request-handling process, nor does it guarantee an environmental response once notifications have been sent.

These temporal-verification results do not contradict the proof obligations discharged in Rodin. Rather, they show that the two verification layers are complementary. Rodin establishes that the invariants are preserved by the main transitions, whereas ProB and LTL reveal that invariant preservation alone is not sufficient to guarantee temporal progress. In this sense, the current model may be correct at the level of state constraints, while still remaining too weak to eliminate incomplete executions, missing recovery branches, or absent environmental responses.

Overall, the experimental results show that the proposed pipeline not only generates Event-B artifacts that can be accepted and processed by Rodin and ProB, but also supports a clear analysis of both the strengths and the limitations of the transformed model. In particular, the current model provides solid support for checking safety properties and invariant preservation, but it still requires strengthening if the objective is to guarantee global progress properties for the Meeting Scheduler process.

8. Related Work

This work lies at the intersection of four research directions: goal-oriented requirements engineering and formalizations of iStar, system-oriented extensions of goal models, Event-B-based approaches to formal specification and verification, and the use of temporal logic for checking safety and progress properties.

Goal-oriented requirements engineering and formalization of iStar.

iStar is one of the most widely used GORE frameworks for modeling stakeholders' intentions, actor dependencies, and the rationale structure of a system [1, 10]. Owing to its expressive representation of goals, tasks, resources, and

dependencies, iStar is particularly useful during early requirements analysis. However, as discussed in the introduction, the core language of iStar remains largely declarative and centered on the actor–goal perspective; it does not directly provide a sufficiently strong formal basis for representing operational behavior in terms of states and transitions, nor for checking temporal properties such as progress and liveness.

To increase the formal rigor of iStar, several works have mapped it to different formal languages. Some studies translate iStar into Z in order to obtain mathematical specifications for analyzing constraints and correctness on the basis of Z's logical foundation [2]. Another line of work uses Alloy to check models and generate counterexamples with respect to goal-model constraints [3]. In addition, ontology- or description-logic-based representations such as iStar2.0-OWL aim to standardize conceptual semantics and support reasoning at the knowledge level [4]. These approaches are valuable for detecting inconsistencies, clarifying structural constraints, and strengthening static analysis. However, they are still primarily focused on structural consistency, type checking, or conceptual reasoning, rather than on providing an end-to-end bridge to an event-based behavioral model that can be analyzed by means of proof obligations and state-space exploration.

System-oriented and adaptive extensions of goal models.

Besides direct formalizations of iStar, many studies have extended goal models in order to better represent adaptation, uncertainty, and domain-specific concerns. For example, FLAGS introduces fuzzy goals to support requirements-driven adaptation [7]. Extensions of iStar/GRL for collaborative cyber-physical systems also enhance modeling capability in complex environments [8]. These works clearly show that richer modeling support is needed when systems operate under changing conditions and stronger environmental constraints.

Nevertheless, increased expressiveness at the modeling level does not automatically imply formal verifiability. In many cases, such extensions still lack a systematic process for normalizing system-oriented information, mapping it to a formal behavioral model, and integrating the overall workflow into a tool-supported verification pipeline. This is precisely the gap addressed in the present work: gStar is not only an extension of iStar at the modeling level, but is also designed to serve as a basis for formalization, automated transformation into Event-B, and verification with Rodin and ProB.

Goal models and Event-B.

Closer to the present work, several studies have used goal-oriented models as input to Event-B specifications. Aziz et al. propose a bridge from goal-oriented requirements to Event-B specifications, showing that goals can be used to guide the construction of formal models [16]. Matoussi et al. present a goal-based approach for guiding the design of an abstract Event-B specification [17]. Ponsard and Devroey continue this line by generating high-level Event-B system models from KAOS requirements models [18]. More recently, Saharudin and Said propose transformation rules from KAOS to Event-B and implement them using ATL in a model-transformation workflow [19].

These works clearly demonstrate the value of connecting goal-oriented requirements to Event-B. However, most Event-B-oriented approaches in this family are based on KAOS rather than iStar, or focus only on part of the target formalization problem. In contrast, the present paper targets an extended fragment of iStar, organized as gStar, and defines a verification-oriented formal semantics for this fragment before mapping it into Event-B. In other words, the contribution is not limited to the generation of target specifications, but also includes an explicit semantic basis for proof, model checking, and temporal analysis.

Event-B, Rodin, ProB, and temporal verification.

Event-B is a state-based formal method centered on invariants, refinement, and proof obligations [5, 20]. Rodin supports the generation and discharge of proof obligations, while ProB supports animation, model checking, deadlock detection, and property checking over Event-B models [5, 6]. This makes Event-B a suitable target platform for connecting normalized requirements models to verifiable formal specifications.

Beyond proof obligations and state-space exploration, Linear Temporal Logic (LTL) provides a classical basis for specifying safety and progress properties over time [9, 12, 13]. However, applying LTL to requirements models is meaningful only when a clear correspondence exists between requirements-side elements and the state variables or events of the behavioral model. In this paper, LTL is not treated as an isolated add-on to the transformation. Instead, it is connected to gStar through a template-guided construction mechanism. This does not replace Event-B proof obligations, but complements them: proof obligations are used to check invariant preservation, whereas LTL is used to analyze temporal safety and progress properties.

Overall, this work differs from iStar-to-Z, iStar-to-Alloy, and iStar-to-OWL approaches in that it does not stop at structural checking or conceptual reasoning, but instead targets an event-based behavioral model analyzable through proof obligations, model checking, and LTL. It also differs from extensions of goal models that only increase conceptual expressiveness, since gStar is explicitly designed to support a concrete formalization and verification workflow. Finally, compared with prior work that uses goal models to guide Event-B specifications, this paper contributes a verification-oriented formal semantics for the considered gStar fragment, an automated gStar to Event-B transformation, and a traceable LTL-based validation layer for analyzing both safety and progress properties.

9. Conclusion and Future Work

This paper has proposed a verification-oriented approach for narrowing the gap between goal-oriented requirements models and formally verifiable specifications. In particular, we extend iStar into gStar in order to strengthen system-aspect modeling and to normalize the information required for formalization. On this basis, the paper develops a verification-oriented formal semantics for the considered gStar fragment, defines a set of semantic mapping rules from gStar to Event-B, and implements these rules within a model-driven engineering pipeline based on Ecore, ATL, and Aceleo.

From a methodological perspective, the contribution does not stop at generating Event-B specifications from requirements models, but also connects the generated artifacts directly to formal analysis activities in Rodin and ProB. In addition, a template-guided mechanism for constructing LTL formulae is proposed in order to support the checking of safety and temporal progress properties while preserving traceability between source-model elements and the properties verified over the target model.

The experimental results on the MeetingScheduler case study show the feasibility of the proposed approach. The generated Event-B models can be imported into and analyzed by Rodin and ProB. At the proof-based verification level, the proof obligations related to invariant preservation for the main events are discharged in Rodin. At the model-checking level, ProB does not report invariant violations in the reported configurations, while also showing that the current model still admits execution branches leading to deadlock. At the temporal-verification level, a key safety property is confirmed, whereas several progress- and response-related LTL formulae yield counterexamples. These counterexamples do not weaken the value of the transformation itself; rather, they make explicit the current behavioral gaps of the model, especially with

respect to process completion, recovery after adverse situations, and environment-dependent response.

Overall, this work contributes in three main respects. First, it provides a system-oriented extension of iStar in the form of gStar. Second, it introduces a verification-oriented formal semantics for the considered gStar fragment together with a semantic mapping into Event-B. Third, it realizes this mapping in a tool-supported pipeline and combines proof obligations, model checking, and LTL-based validation to analyze the transformed model in a systematic manner.

Several directions remain for future work. An important direction is to enrich the semantic scope and the mapping coverage so as to include additional gStar/iStar constructs beyond those considered in the present paper. Another direction is to strengthen the progress-related constraints in the target model in order to reduce temporal counterexamples and limit incomplete execution branches. In addition, deeper automation of the LTL construction process, together with evaluation on larger and more diverse case studies, would provide further evidence of the generality and practical usefulness of the approach. Finally, the semantic-preservation statements formulated in this paper could be further developed into more rigorous formal proofs for broader fragments of gStar.

References

- [1] F. Dalpiaz, X. Franch, J. Horkoff, iStar 2.0 language guide (2016). *arXiv:1605.07767*, doi:10.48550/arXiv.1605.07767.
URL <https://arxiv.org/abs/1605.07767>
- [2] L. Kadakolmath, U. D. Ramu, iStar goal model to Z formal model translation and model checking of CBTC moving block interlocking system, *Formal Aspects of Computing*, Vol. 36, No. 1, 2024, pp. 6:1–6:45. doi: 10.1145/3633065.
URL <https://doi.org/10.1145/3633065>
- [3] P. O. Ating'a, A. Krishna, Verification of i* models using alloy, in: *Information Systems Development: Challenges in Practice, Theory, and Education*, 2010, pp. 63–74.
URL <https://dblp.org/rec/conf/isdevel/AtingaK10>
- [4] C. Almendra, C. Silva, V. E. S. Souza, R. Guizzardi, C. Bernabé, iStar2.0-OWL: an operational ontology for iStar, in: *Proceedings of the Twelfth International i* Workshop (iStar 2019)*, Vol. 2490 of *CEUR Workshop Proceedings*, 2019.
URL <https://ceur-ws.org/Vol-2490/paper2.pdf>
- [5] J.-R. Abrial, M. Butler, S. Hallerstede, T. S. Hoang, F. Mehta, L. Voisin, Rodin: An open toolset for modelling and reasoning in Event-B, *Software Tools for Technology Transfer*, Vol. 12, No. 6, 2010, pp. 447–466. doi:10.1007/s10009-010-0145-y.
URL <https://doi.org/10.1007/s10009-010-0145-y>
- [6] M. Leuschel, M. J. Butler, Prob: A model checker for B, in: *FME 2003: Formal Methods*, Vol. 2805 of *Lecture Notes in Computer Science*, 2003, pp. 855–874.
URL <https://dblp.org/rec/conf/fm/LeuschelB03>
- [7] L. Baresi, L. Pasquale, P. Spoletini, Fuzzy goals for requirements-driven adaptation, in: *Proceedings of the 18th IEEE International Requirements Engineering Conference (RE 2010)*, 2010. doi:10.1109/RE.2010.25.
URL <https://doi.org/10.1109/RE.2010.25>
- [8] M. Daun, J. Brings, L. Krajinski, V. Stenkova, T. Bandyszak, A GRL-compliant iStar extension for collaborative cyber-physical systems, *Requirements Engineering*, Vol. 26, No. 3, 2021, pp. 325–370. doi:10.1007/s00766-021-00347-3.
URL <https://doi.org/10.1007/s00766-021-00347-3>
- [9] A. Pnueli, The temporal logic of programs, in: *18th Annual Symposium on Foundations of Computer Science (FOCS 1977)*, 1977, pp. 46–57. doi:10.1109/SFCS.1977.32.
URL <https://doi.org/10.1109/SFCS.1977.32>
- [10] E. S. K. Yu, Towards modelling and reasoning support for early-phase requirements engineering, in: *Proceedings of the 3rd IEEE International Symposium on Requirements Engineering (RE '97)*, IEEE Computer Society, 1997, pp. 226–235. doi: 10.1109/ISRE.1997.566873.
- [11] J.-R. Abrial, *Modeling in Event-B: System and Software Engineering*, Cambridge University Press, 2010. doi:10.1017/CB09781139195881.
URL <https://dblp.org/rec/books/daglib/0024570.html>
- [12] C. Baier, J.-P. Katoen, *Principles of Model Checking*, MIT Press, 2008.

- [13] E. M. Clarke, O. Grumberg, D. A. Peled, Model Checking, MIT Press, 1999.
- [14] A. van Lamsweerde, E. Letier, Handling obstacles in goal-oriented requirements engineering, IEEE Transactions on Software Engineering, Vol. 26, No. 10, 2000, pp. 978–1005. doi:10.1109/32.879820. URL <https://doi.org/10.1109/32.879820>
- [15] A. van Lamsweerde, Requirements Engineering: From System Goals to UML Models to Software Specifications, Wiley, 2009.
- [16] B. Aziz, A. Arenas, J. Bicarregui, C. Ponsard, P. Massonet, From goal-oriented requirements to event-b specifications, in: NASA Formal Methods, Lecture Notes in Computer Science, 2009, pp. 96–105. URL <https://dblp.org/rec/conf/nfm/AzizABPM09>
- [17] A. Matoussi, F. Gervais, R. Laleau, A goal-based approach to guide the design of an abstract event-b specification, in: 16th IEEE International Conference on Engineering of Complex Computer Systems (ICECCS 2011), 2011, pp. 139–148. doi:10.1109/ICECCS.2011.21. URL <https://dblp.org/rec/conf/iceccs/MatoussiGL11>
- [18] C. Ponsard, X. Devroey, Generating high-level event-b system models from kaos requirements models, in: INFORSID 2011, 2011, pp. 317–332. URL <https://dblp.org/rec/conf/inforsid/PonsardD11>
- [19] S. N. Saharudin, M. Y. Said, Keep all objectives satisfied (kaos) to event-b models transformation, International Journal on Advanced Science, Engineering and Information Technology, Vol. 12, No. 6, 2022, pp. 2443–2451. doi:10.18517/ijaseit.12.6.15875. URL <https://ijaseit.insightsociety.org/index.php/ijaseit/article/view/15875>
- [20] J.-R. Abrial, Modeling in Event-B: System and Software Engineering, Cambridge University Press, 2010. doi:10.1017/CB09781139195881. URL <https://dblp.org/rec/books/daglib/0024570.html>